

Loja Online - Relatorio Tecnico da Fase V

Casamento de padroes com KMP/Boyer-Moore e criptografia XOR.

1. Link para o GitHub

Repositorio: <https://github.com/lucaszanfa/Tp-Aedes3>

Video explicativo da Fase V: https://youtu.be/dXa_AAKLjUk

2. Atualizacao do README

O arquivo README.md foi atualizado com o link do repositorio, instrucoes de compilacao e execucao, descricao da rota /pesquisa, descricao dos algoritmos KMP e Boyer-Moore e descricao da criptografia XOR aplicada ao campo Cliente.email.

3. Objetivo da fase

Esta fase acrescenta ao sistema dois algoritmos de casamento de padroes, KMP e Boyer-Moore, e criptografia XOR em um campo sensivel. Os recursos foram integrados ao aplicativo web local preservando a arquitetura MVC + DAO, os arquivos binarios, os indices e os CRUDs existentes.

4. Arquivos principais

Arquivo	Responsabilidade
Util/PatternMatcher.java	Implementa KMP e Boyer-Moore.
Controller/PesquisaController.java	Aplica o algoritmo escolhido sobre os produtos.
Main/App.java	Cria a rota /pesquisa, recebe o formulario e exibe resultados.
View/HtmlView.java	Inclui a opcao de menu "Pesquisar por padrao (KMP / BM)".

Util/XorCipher.java	Implementa a criptografia e descriptografia XOR.
Model/Cliente.java	Criptografa o email ao salvar e descriptografa ao ler.

5. Interface de pesquisa

A interface fica disponivel em <http://localhost:18080/pesquisa>. Ela permite escolher KMP ou Boyer-Moore, informar o padrao pesquisado e visualizar os registros encontrados.

```
String algoritmo =  
pesquisaController.normalizeAlgorithm(data.get("algoritmo"));  
String padrao = data.get("padrao");  
List<Produto> encontrados =  
pesquisaController.pesquisarProdutosPorNome(algoritmo, padrao);
```

6. Formulario tecnico

1. Qual campo textual foi escolhido para aplicar os algoritmos de casamento de padroes? Por que?

O campo escolhido foi nome da entidade Produto. Ele foi escolhido porque e textual, aparece naturalmente no catalogo da loja e representa uma consulta real para o usuario. Em uma loja online, buscar produtos por parte do nome e uma funcionalidade comum.

2. Explique o funcionamento do KMP implementado

O KMP esta no metodo containsKmp da classe Util.PatternMatcher. Primeiro, o algoritmo cria a tabela LPS pelo metodo buildLongestPrefixSuffix. Essa tabela guarda o tamanho do maior prefixo que tambem e sufixo em cada ponto do padrao.

Depois, o texto e percorrido com o indice i e o padrao com o indice j. Quando os caracteres coincidem, os dois avancam. Quando ha divergencia, o texto nao volta; se $j > 0$, o padrao volta para $lps[j - 1]$. Se $j == 0$, apenas o texto avanca.

```
j = lps[j - 1];
```

3. Explique o funcionamento do Boyer-Moore implementado

O Boyer-Moore esta no metodo `containsBoyerMoore` da classe `Util.PatternMatcher`. A versao implementada usa a heuristica obrigatoria `bad character`. O metodo `buildBadCharacterTable` cria uma tabela com a ultima ocorrencia de cada caractere no padrao.

A comparacao ocorre da direita para a esquerda. Quando ha divergencia, o algoritmo calcula o deslocamento com base no caractere que causou o erro, garantindo pelo menos uma posicao de avanco.

```
shift += Math.max(1, j - badChar[text.charAt(shift + j)]);
```

A heuristica `good suffix` nao foi usada porque era opcional no enunciado.

4. Descreva como integrou os algoritmos ao sistema.

A integracao foi feita em tres camadas. Em `Util.PatternMatcher` ficam os algoritmos. Em `Controller.PesquisaController` fica a regra de pesquisa, que percorre os produtos e aplica o algoritmo sobre `Produto.nome`. Em `Main.App` e `View.HtmlView` fica a interface web.

```
boolean match = "BM".equals(normalizedAlgorithm)
    ? PatternMatcher.containsBoyerMoore(nome, normalizedPattern)
    : PatternMatcher.containsKmp(nome, normalizedPattern);
```

5. Quais dificuldades encontrou na implementacao dos dois algoritmos?

No KMP, a principal dificuldade foi construir corretamente a tabela LPS para tratar prefixos e sufixos parciais. No Boyer-Moore, o cuidado principal foi montar a tabela `bad character` e garantir que o deslocamento nunca fosse zero ou negativo. Outra dificuldade foi integrar os algoritmos sem quebrar o padrao MVC, mantendo algoritmos em `Util`, regra de pesquisa em `Controller` e interface na camada web.

6. Qual campo foi utilizado na criptografia?

O campo utilizado foi `email` da entidade `Cliente`, por ser um dado pessoal e sensivel.

7. Qual foi o metodo utilizado na criptografia?

Foi usada criptografia XOR, implementada em Util.XorCipher. O texto e convertido para bytes em UTF-8, cada byte e combinado com a chave usando o operador XOR (^) e o resultado e salvo em Base64 com o prefixo XOR1:.

```
output[i] = (byte) (input[i] ^ KEY[i % KEY.length]);
```

Na gravacao do cliente:

```
values[1] = XorCipher.encrypt(email);
```

Na leitura do cliente:

```
email = values.length > 1 ? XorCipher.decryptIfEncrypted(values[1]) : "";
```

7. Como compilar e executar

Compilar:

```
New-Item -ItemType Directory -Force out\classes | Out-Null
$sources = Get-ChildItem -Recurse -Filter *.java -Path
Controller,DAO,Main,Model,Util,View | ForEach-Object { $_.FullName }
javac --release 8 -d out\classes $sources
```

Executar:

```
java -cp out\classes Main.App
```

Abrir <http://localhost:18080> ou diretamente <http://localhost:18080/pesquisa>.

8. Video explicativo

Video da Fase V: https://youtu.be/dXa_AAKLjUk

O video explicativo mostra Util/PatternMatcher.java, Controller/PesquisaController.java, a rota /pesquisa, Util/XorCipher.java e a serializacao do email em Model/Cliente.java.

9. Conclusao

A fase foi integrada ao aplicativo com os dois algoritmos de casamento de padroes pedidos e com criptografia XOR em campo sensivel. A busca usa o campo Produto.nome, a interface permite escolher entre KMP e Boyer-Moore, e os registros encontrados sao retornados ao usuario pela tela web. A criptografia foi aplicada ao email do cliente durante a persistencia binaria, com descriptografia automatica na leitura.